

Day 2：后端项目架构初始化 — 详细教程

目录

- [一、今日目标与验收标准](#)
- [二、后端目录结构设计与模块划分](#)
- [三、全局配置模块 \(config\)](#)
- [四、数据库连接模块 \(database\)](#)
- [五、SQLAlchemy 数据模型定义 \(entity/db_models.py\)](#)
- [六、Pydantic 请求/响应模型 \(entity/schemas.py\)](#)
- [七、Alembic 数据库迁移](#)
- [八、MinIO 存储客户端 \(storage\)](#)
- [九、Docker Compose 基础设施搭建](#)
- [十、认证模块实现 \(JWT 登录/注册\)](#)
- [十一、main.py 入口完善与路由注册](#)
- [十二、验证与测试](#)
- [十三、常见问题排查](#)

一、今日目标与验收标准

1.1 今日任务总览

时间段	内容	形式
上午 0.5h	后端目录结构讲解、模块划分原则	讲授
上午 1h	main.py 入口、config 配置模块、database 数据库连接	实操
上午 0.5h	SQLAlchemy 数据模型定义	实操
上午 0.5h	Alembic 数据库迁移工具使用	实操
上午 0.5h	Pydantic 请求/响应模型	实操
下午 1h	Docker Compose：PostgreSQL + Redis + MinIO	实操
下午 0.5h	验证数据库/Redis/MinIO 连接	实操
下午 1h	认证模块：JWT 登录/注册 API	实操
下午 0.5h	Postman/Apifox 测试 API	实操

1.2 验收标准

- ☐ `docker compose up -d` 可启动全部基础设施 (PostgreSQL、Redis、MinIO)
- ☐ 后端服务可启动, 访问 `http://localhost:8000/docs` 可看到 Swagger 文档
- ☐ 注册 API 可正常创建用户
- ☐ 登录 API 可正常返回 JWT Token
- ☐ MinIO 控制台 `http://localhost:9001` 可访问

二、后端目录结构设计与模块划分

2.1 目标目录结构

Day 2 结束后, backend 目录应如下:

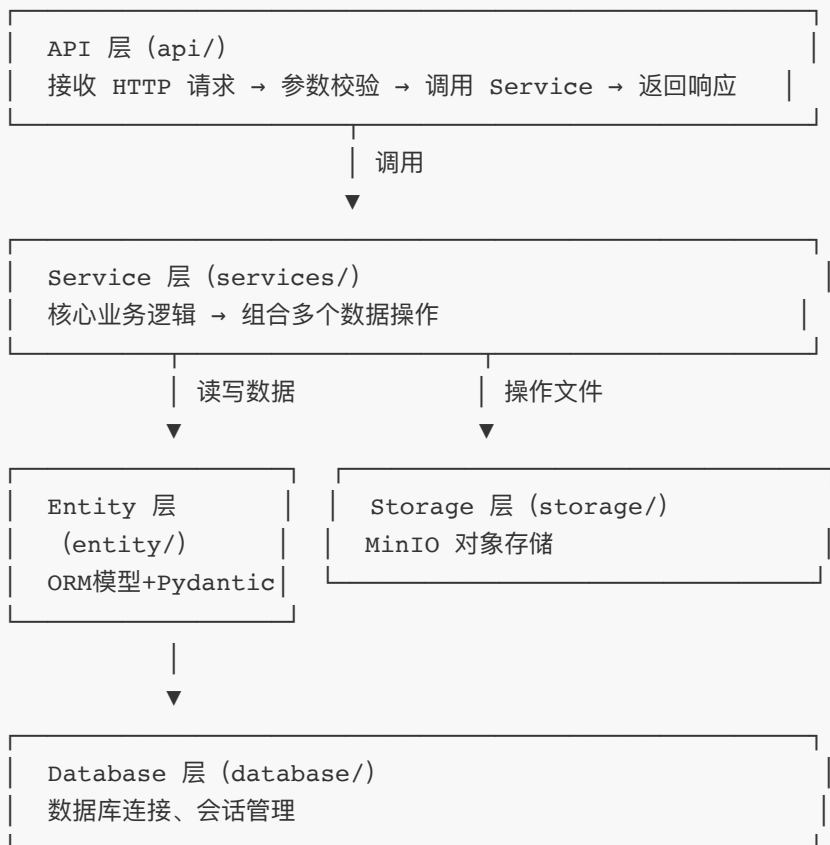
```
backend/
├── main.py                # 应用入口 (已有, 需完善)
├── requirements.txt       # Python 依赖 (已有)
├── .env                  # 环境变量 (从 .env.example 复制)
├── .env.example          # 环境变量模板 (已有)
├── alembic.ini           # Alembic 配置文件
├── alembic/              # Alembic 迁移目录
│   ├── env.py
│   └── versions/
└── app/
    ├── __init__.py
    ├── config/            # 配置模块
    │   ├── __init__.py
    │   ├── settings.py    # 全局配置 (数据库、Redis、JWT 等)
    │   ├── detection.py   # 检测配置 (Day 8 用)
    │   └── llm.py         # LLM 配置 (Day 11 用)
    ├── database/          # 数据库模块
    │   ├── __init__.py
    │   └── session.py     # 数据库引擎、会话、Base
    ├── entity/            # 数据模型
    │   ├── __init__.py
    │   ├── db_models.py   # SQLAlchemy ORM 模型
    │   └── schemas.py     # Pydantic 请求/响应模型
    ├── storage/           # 存储模块
    │   ├── __init__.py
    │   └── minio_client.py # MinIO 客户端封装
    ├── services/          # 业务服务层
    │   └── __init__.py
```

```

├── user_service.py      # 用户服务（注册、登录、鉴权）
├── core/                # 核心工具
│   ├── __init__.py
│   └── security.py      # JWT 生成/验证、密码加密
├── api/                 # API 路由层
│   ├── __init__.py
│   └── auth.py          # 认证 API（登录/注册）

```

2.2 模块划分原则



分层原则：

- **API 层**只负责 HTTP 交互，不写业务逻辑
- **Service 层**封装业务逻辑，可被多个 API 复用
- **Entity 层**定义数据结构，分为数据库模型和 API 模型
- **Database 层**只负责数据库连接，不包含业务逻辑

2.3 创建目录结构

在终端中执行以下命令：

```
cd backend
```

```
# 创建所有子目录
mkdir -p app/config
mkdir -p app/database
mkdir -p app/entity
mkdir -p app/storage
mkdir -p app/services
mkdir -p app/core
mkdir -p app/api

# 创建所有 __init__.py (Python 包标识文件)
touch app/config/__init__.py
touch app/database/__init__.py
touch app/entity/__init__.py
touch app/storage/__init__.py
touch app/services/__init__.py
touch app/core/__init__.py
touch app/api/__init__.py
```

三、全局配置模块 (config)

3.1 创建 settings.py

文件: `backend/app/config/settings.py`

```
"""
全局配置模块
使用 pydantic-settings 管理所有配置项，支持从 .env 文件和环境变量读取
加载优先级：环境变量（系统级别）> .env 文件 > 代码中的默认值
"""

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    """应用全局配置"""

    # — 应用基础配置 —
    APP_NAME: str = "RSOD Agent Platform"
    APP_VERSION: str = "0.1.0"
    DEBUG: bool = True
    LOG_LEVEL: str = "INFO"

    # — 数据库配置 —
    DB_HOST: str = "localhost"
    DB_PORT: int = 5432
    DB_NAME: str = "rsod_agent"
    DB_USER: str = "rsod_admin"
    DB_PASSWORD: str = "rsod_admin"

    @property
```

```

def DATABASE_URL(self) -> str:
    """构造 PostgreSQL 连接字符串"""
    return f"postgresql://{self.DB_USER}:{self.DB_PASSWORD}@{self.DB_HOST}:{self.DB_PORT}/{self.DB_NAME}"

# — Redis 配置 —
REDIS_HOST: str = "localhost"
REDIS_PORT: int = 6379

@property
def REDIS_URL(self) -> str:
    """构造 Redis 连接字符串"""
    return f"redis://{self.REDIS_HOST}:{self.REDIS_PORT}/0"

# — MinIO 配置 —
MINIO_ENDPOINT: str = "localhost:9000"
MINIO_ACCESS_KEY: str = "minioadmin"
MINIO_SECRET_KEY: str = "minioadmin"
MINIO_BUCKET: str = "rsod-agent-images"
MINIO_SECURE: bool = False

# — JWT 认证配置 —
JWT_SECRET_KEY: str = "your-super-secret-key-change-in-production"
JWT_ALGORITHM: str = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES: int = 30

# — CORS 配置 —
ALLOWED_ORIGINS: str = (
    "http://localhost:3000,http://localhost:5173,http://localhost:8080"
)

@property
def cors_origins_list(self) -> list:
    """将 CORS 配置字符串转为列表"""
    return [origin.strip() for origin in self.ALLOWED_ORIGINS.split(",")]

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

# 创建全局单例，其他模块直接 import 使用
settings = Settings()

```

关键知识点讲解：

1. **pydantic-settings**: 继承 `BaseSettings`，自动从环境变量和 `.env` 文件读取配置
2. **@property**: 动态计算属性，如 `DATABASE_URL` 由多个字段拼接而成
3. **全局单例**: `settings = Settings()` 在模块级别实例化，整个应用共享同一份配置

3.2 创建 .env 文件

```
cd backend
cp .env.example .env
```

然后编辑 `.env`，将数据库配置改为与 `docker-compose.yml` 一致的值：

```
# 数据库配置（与 docker-compose.yml 中的环境变量对应）
DB_HOST=localhost
DB_PORT=5432
DB_NAME=rsod_agent
DB_USER=rsod_admin
DB_PASSWORD=rsod_admin

# Redis 配置
REDIS_HOST=localhost
REDIS_PORT=6379

# MinIO 配置
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET=rsod-agent-images
MINIO_SECURE=false

# JWT 配置
JWT_SECRET_KEY=rsod-dev-secret-key-2026
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# CORS
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173,http://localhost:8080
```

注意：`.env` 文件已在 `.gitignore` 中，不会被提交到 Git。团队协作时，每人各自创建自己的 `.env`。

四、数据库连接模块（database）

4.1 创建 session.py

文件：`backend/app/database/session.py`

```
"""
数据库连接与会话管理
- 创建 SQLAlchemy 引擎和会话工厂
- 提供 get_db 依赖注入函数，供 API 层使用
"""

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

from app.config.settings import settings
```

```

# 创建数据库引擎
# pool_pre_ping=True: 每次从连接池取连接前先测试是否可用
engine = create_engine(
    settings.DATABASE_URL,
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True,
    echo=settings.DEBUG, # DEBUG 模式下打印 SQL 语句
)

# 会话工厂
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
)

# ORM 模型的基础类, 所有模型都继承自它
Base = declarative_base()

def get_db():
    """
    获取数据库会话的依赖注入函数
    在 FastAPI 路由中通过 Depends(get_db) 使用

    用法示例:
        @router.get("/xxx")
        def my_api(db: Session = Depends(get_db)):
            ...
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

关键知识点讲解:

1. **engine**: 数据库连接池管理器, `pool_size=10` 表示保持 10 个连接
2. **SessionLocal**: 会话工厂, 每次调用 `SessionLocal()` 创建一个新的数据库会话
3. **Base**: 所有 ORM 模型的父类, Alembic 迁移时通过它发现所有模型
4. **get_db**: FastAPI 依赖注入模式, 使用 `yield` 确保请求结束后自动关闭连接

五、SQLAlchemy 数据模型定义 (entity/db_models.py)

5.1 数据模型设计思路

用户与权限 (RBAC)
users ← user_roles → roles ← role_permissions → permissions

检测业务
detection_scenes (场景)
detection_tasks (任务) → detection_results (目标级结果)

模型管理
training_tasks (训练任务) → training_metrics (逐epoch指标)
model_versions (模型版本管理)

智能体对话
chat_sessions (会话) → chat_messages (消息)

系统运维
operation_logs (操作审计日志)

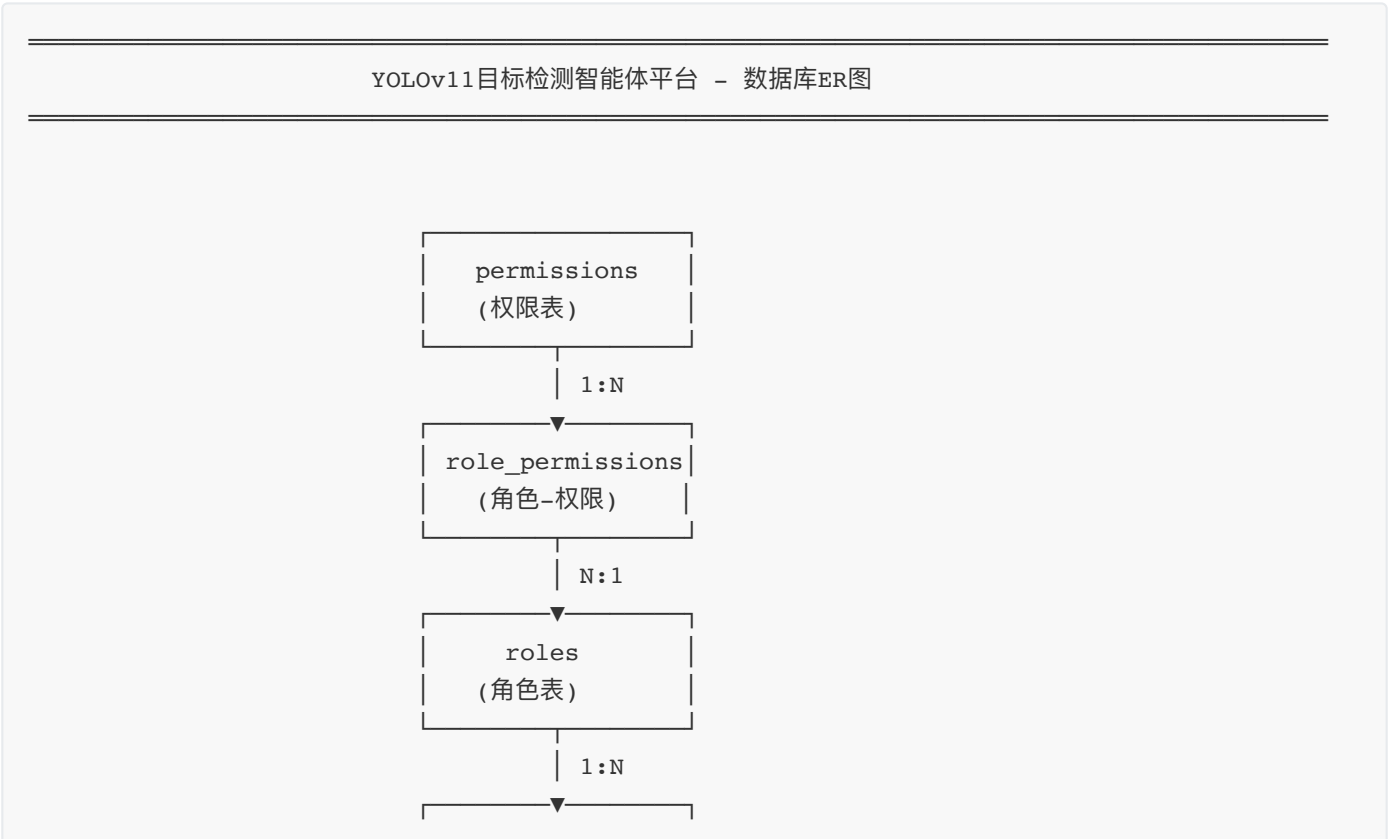
5.2 数据库表设计总览（15 张表）

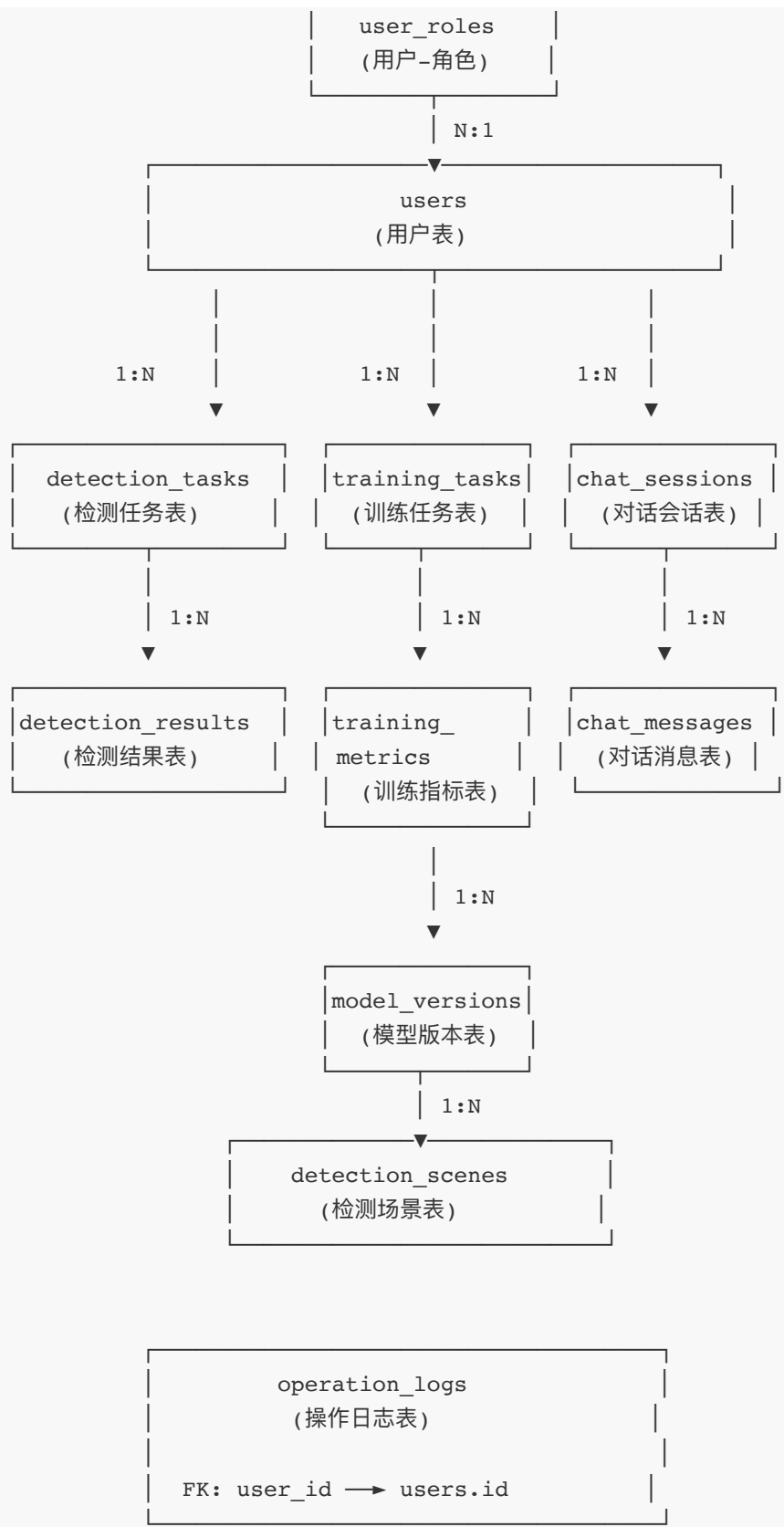
模块	表名	职责
用户权限	users	用户基本信息
	roles	角色定义
	permissions	权限定义（细粒度）
	user_roles	用户↔角色 多对多
	role_permissions	角色↔权限 多对多
检测业务	detection_scenes	检测场景（遥感/工业/农业...）
	detection_tasks	检测任务（每次操作一条）
	detection_results	目标级结果（每个目标一条）
模型管理	training_tasks	训练任务
	training_metrics	逐 epoch 训练指标
	model_versions	模型版本管理
智能体	chat_sessions	对话会话
	chat_messages	对话消息（含工具调用）
系统运维	operation_logs	操作审计日志

关键设计决策说明

设计点	方案	为什么这样做
RBAC 权限	users → user_roles → roles → role_permissions → permissions	企业标准五表模型，支持细粒度权限控制，角色可灵活组合
检测场景	独立 detection_scenes 表	每个小组/业务方向一个场景，隔离模型和类别，便于多租户
检测结果	detection_tasks + detection_results 双层	任务级存统计，结果级存每个目标，支持按类别筛选/统计
模型版本	独立 model_versions 表	支持训练产出 + 手动上传两种来源，支持版本切换和回滚
训练指标	独立 training_metrics 表	每 epoch 一条记录，前端可直接查询绘制实时曲线
智能体对话	chat_sessions + chat_messages	会话级管理历史，消息级记录工具调用链路，支持审计
操作日志	operation_logs	企业级审计需求，记录谁在什么时间做了什么操作
JSON 字段	class_names、bbox、tool_calls 等用 JSON	灵活存储结构化数据，避免过度拆表

5.3 数据模型设计概览





表关系清单：

- | | |
|---------------------------------|------------|
| 1. users —1:N→ user_roles | 用户可拥有多个角色 |
| 2. roles —1:N→ user_roles | 角色可被多个用户拥有 |
| 3. roles —1:N→ role_permissions | 角色可拥有多个权限 |

4. permissions —1:N→ role_permissions	权限可被多个角色拥有
5. users —1:N→ detection_tasks	用户可创建多个检测任务
6. detection_scenes —1:N→ detection_tasks	场景下有多个检测任务
7. detection_tasks —1:N→ detection_results	检测任务包含多个检测结果
8. model_versions —1:N→ detection_tasks	模型版本可用于多个检测任务
9. users —1:N→ training_tasks	用户可创建多个训练任务
10. detection_scenes —1:N→ training_tasks	场景下有多个训练任务
11. training_tasks —1:N→ training_metrics	训练任务包含多个epoch指标
12. training_tasks —1:N→ model_versions	训练任务产出多个模型版本
13. detection_scenes —1:N→ model_versions	场景下有多个模型版本
14. users —1:N→ chat_sessions	用户可拥有多个对话会话
15. chat_sessions —1:N→ chat_messages	会话包含多条消息
16. users —1:N→ operation_logs	用户有多个操作日志

表数量：15张

模块划分：用户权限(5) | 检测业务(3) | 模型管理(3) | 智能体(2) | 系统运维(1)

5.4 创建 db_models.py

文件：backend/app/entity/db_models.py

"""

数据库模型定义

表结构总览：

用户权限: users, roles, permissions, user_roles, role_permissions

检测业务: detection_scenes, detection_tasks, detection_results

模型管理: training_tasks, training_metrics, model_versions

智能体: chat_sessions, chat_messages

系统运维: operation_logs

"""

from datetime import datetime

from sqlalchemy import (
 Column, Integer, String, Float, DateTime, ForeignKey,
 JSON, Text, Boolean, Enum, Table, BigInteger
)

from sqlalchemy.orm import relationship

from app.database.session import Base

=====
 # 一、用户与权限 (RBAC)
 # =====

class User(Base):
 """用户表"""
 __tablename__ = "users"

```

id = Column(Integer, primary_key=True, autoincrement=True)
username = Column(String(50), unique=True, nullable=False, index=True, comment="用户名")
email = Column(String(100), unique=True, nullable=False, index=True, comment="邮箱")
hashed_password = Column(String(255), nullable=False, comment="加密密码")
phone = Column(String(20), nullable=True, comment="手机号")
avatar = Column(String(500), nullable=True, comment="头像 URL")
is_active = Column(Boolean, default=True, comment="是否启用")
is_superuser = Column(Boolean, default=False, comment="是否超级管理员")
last_login_at = Column(DateTime, nullable=True, comment="最后登录时间")
created_at = Column(DateTime, default=datetime.now, comment="创建时间")
updated_at = Column(DateTime, default=datetime.now, onupdate=datetime.now, comment="更新时间")

```

关联

```

user_roles = relationship("UserRole", back_populates="user", cascade="all, delete-orphan")
detection_tasks = relationship("DetectionTask", back_populates="user")
training_tasks = relationship("TrainingTask", back_populates="user")
chat_sessions = relationship("ChatSession", back_populates="user")
operation_logs = relationship("OperationLog", back_populates="user")

```

```
class Role(Base):
```

"""角色表"""

```
__tablename__ = "roles"
```

```

id = Column(Integer, primary_key=True, autoincrement=True)
name = Column(String(50), unique=True, nullable=False, comment="角色标识, 如 admin/operator/viewer")
display_name = Column(String(100), nullable=False, comment="角色显示名, 如 管理员/操作员/访客")
description = Column(String(500), nullable=True, comment="角色描述")
is_system = Column(Boolean, default=False, comment="是否系统内置角色 (不可删除)")
created_at = Column(DateTime, default=datetime.now, comment="创建时间")

```

关联

```

user_roles = relationship("UserRole", back_populates="role", cascade="all, delete-orphan")
role_permissions = relationship("RolePermission", back_populates="role", cascade="all, delete-orphan")

```

```
class Permission(Base):
```

"""权限表"""

```
__tablename__ = "permissions"
```

```

id = Column(Integer, primary_key=True, autoincrement=True)
code = Column(String(100), unique=True, nullable=False, comment="权限编码, 如 detection:task:create")
name = Column(String(100), nullable=False, comment="权限名称")

```

```

    module = Column(String(50), nullable=False, comment="所属模块:
auth/detection/training/agent/system")
    description = Column(String(500), nullable=True, comment="权限描述")

# 关联
    role_permissions = relationship("RolePermission", back_populates="permission",
cascade="all, delete-orphan")

class UserRole(Base):
    """用户-角色关联表 (多对多) """
    __tablename__ = "user_roles"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    role_id = Column(Integer, ForeignKey("roles.id"), nullable=False, index=True)
    created_at = Column(DateTime, default=datetime.now)

    user = relationship("User", back_populates="user_roles")
    role = relationship("Role", back_populates="user_roles")

class RolePermission(Base):
    """角色-权限关联表 (多对多) """
    __tablename__ = "role_permissions"

    id = Column(Integer, primary_key=True, autoincrement=True)
    role_id = Column(Integer, ForeignKey("roles.id"), nullable=False, index=True)
    permission_id = Column(Integer, ForeignKey("permissions.id"), nullable=False,
index=True)

    role = relationship("Role", back_populates="role_permissions")
    permission = relationship("Permission", back_populates="role_permissions")

# =====
# 二、检测业务
# =====

class DetectionScene(Base):
    """检测场景配置表
    每个小组/业务方向一个场景，如：遥感检测、工业缺陷、农业病害等
    场景决定了使用哪个模型、检测哪些类别
    """
    __tablename__ = "detection_scenes"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(100), unique=True, nullable=False, comment="场景标识，如
remote_sensing")
    display_name = Column(String(100), nullable=False, comment="场景显示名，如 遥感目标检测")
    description = Column(Text, nullable=True, comment="场景描述")

```

```

    category = Column(String(50), nullable=False, comment="场景分类:
agriculture/industry/remote_sensing/medical/traffic")
    class_names = Column(JSON, nullable=False, comment="类别列表, 如
[\"airplane\", \"storage-tank\"]")
    class_names_cn = Column(JSON, nullable=True, comment="类别中文名映射, 如
{\"airplane\": \"飞机\"}")
    is_active = Column(Boolean, default=True, comment="是否启用")
    created_by = Column(Integer, ForeignKey("users.id"), nullable=True, comment="创建人")
    created_at = Column(DateTime, default=datetime.now)
    updated_at = Column(DateTime, default=datetime.now, onupdate=datetime.now)

# 关联
detection_tasks = relationship("DetectionTask", back_populates="scene")
model_versions = relationship("ModelVersion", back_populates="scene")
training_tasks = relationship("TrainingTask", back_populates="scene")

class DetectionTask(Base):
    """检测任务表 - 每次检测操作生成一条任务记录"""
    __tablename__ = "detection_tasks"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True,
comment="操作用户")
    scene_id = Column(Integer, ForeignKey("detection_scenes.id"), nullable=False,
index=True, comment="使用的检测场景")
    model_version_id = Column(Integer, ForeignKey("model_versions.id"), nullable=True,
comment="使用的模型版本")
    task_type = Column(String(20), nullable=False, comment="检测类型:
single/batch/folder/video/camera")
    status = Column(String(20), default="pending", comment="状态:
pending/processing/completed/failed")

# 检测统计
total_images = Column(Integer, default=0, comment="处理图像总数")
total_objects = Column(Integer, default=0, comment="检测到目标总数")
total_inference_time = Column(Float, default=0, comment="总推理耗时 (ms)")

# 检测参数
conf_threshold = Column(Float, default=0.25, comment="置信度阈值")
iou_threshold = Column(Float, default=0.45, comment="NMS IoU 阈值")
image_size = Column(Integer, default=640, comment="推理图像尺寸")

# 错误信息
error_message = Column(Text, nullable=True, comment="失败时的错误信息")

# 分析与建议 (AI 生成)
analysis_report = Column(Text, nullable=True, comment="分析报告 (Markdown 格式)")
analysis_suggestion = Column(Text, nullable=True, comment="专业建议")
risk_level = Column(String(20), nullable=True, comment="风险等级:
low/medium/high/critical")
analyzed_at = Column(DateTime, nullable=True, comment="分析完成时间")

```

```

created_at = Column(DateTime, default=datetime.now, index=True, comment="创建时间")
completed_at = Column(DateTime, nullable=True, comment="完成时间")

# 关联
user = relationship("User", back_populates="detection_tasks")
scene = relationship("DetectionScene", back_populates="detection_tasks")
model_version = relationship("ModelVersion", back_populates="detection_tasks")
results = relationship("DetectionResult", back_populates="task", cascade="all, delete-orphan")

class DetectionResult(Base):
    """检测结果表 - 每张图像中每个检测到的目标一条记录"""
    __tablename__ = "detection_results"

    id = Column(Integer, primary_key=True, autoincrement=True)
    task_id = Column(Integer, ForeignKey("detection_tasks.id"), nullable=False,
index=True, comment="所属检测任务")
    image_path = Column(String(500), nullable=False, comment="原始图像路径")
    annotated_image_url = Column(String(500), nullable=True, comment="标注图像 MinIO URL")

    # 单个目标信息
    class_name = Column(String(50), nullable=False, index=True, comment="类别名称")
    class_name_cn = Column(String(50), nullable=True, comment="类别中文名")
    class_id = Column(Integer, nullable=False, comment="类别 ID")
    confidence = Column(Float, nullable=False, comment="置信度 0~1")
    bbox = Column(JSON, nullable=False, comment="边界框 [x1, y1, x2, y2]")

    # 图像级信息 (冗余存储, 方便查询)
    inference_time = Column(Float, nullable=True, comment="该图推理耗时 (ms)")
    image_width = Column(Integer, nullable=True, comment="图像宽度")
    image_height = Column(Integer, nullable=True, comment="图像高度")

    created_at = Column(DateTime, default=datetime.now)

    # 关联
    task = relationship("DetectionTask", back_populates="results")

# =====
# 三、模型管理
# =====

class TrainingTask(Base):
    """模型训练任务表"""
    __tablename__ = "training_tasks"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True,
comment="操作用户")

```

```

    scene_id = Column(Integer, ForeignKey("detection_scenes.id"), nullable=False,
index=True, comment="关联场景")
    task_uuid = Column(String(100), unique=True, nullable=False, index=True, comment="任务
唯一标识")
    status = Column(String(20), default="pending", comment="状态:
pending/running/completed/failed/cancelled")

# 训练配置
    model_name = Column(String(50), default="yolov11n", comment="基础模型:
yolov11n/s/m/l/x")
    epochs = Column(Integer, default=100, comment="训练轮数")
    img_size = Column(Integer, default=640, comment="图像尺寸")
    batch_size = Column(Integer, default=16, comment="批次大小")
    device = Column(String(20), default="0", comment="训练设备: 0/1/cpu")
    optimizer = Column(String(20), default="SGD", comment="优化器: SGD/Adam/AdamW")
    lr0 = Column(Float, default=0.01, comment="初始学习率")
    augment_config = Column(JSON, nullable=True, comment="数据增强配置")

# 训练进度
    current_epoch = Column(Integer, default=0, comment="当前轮数")
    progress = Column(Integer, default=0, comment="进度百分比 0~100")

# 数据集信息
    dataset_path = Column(String(500), nullable=True, comment="数据集路径")
    dataset_size = Column(Integer, nullable=True, comment="数据集图像数量")
    data_yaml = Column(String(500), nullable=True, comment="data.yaml 路径")

# 错误信息
    error_message = Column(Text, nullable=True, comment="失败错误信息")

    created_at = Column(DateTime, default=datetime.now, comment="创建时间")
    updated_at = Column(DateTime, default=datetime.now, onupdate=datetime.now, comment="更
新时间")
    started_at = Column(DateTime, nullable=True, comment="开始训练时间")
    completed_at = Column(DateTime, nullable=True, comment="训练完成时间")

# 关联
    user = relationship("User", back_populates="training_tasks")
    scene = relationship("DetectionScene", back_populates="training_tasks")
    metrics = relationship("TrainingMetric", back_populates="task", cascade="all, delete-
orphan")
    model_versions = relationship("ModelVersion", back_populates="training_task")

class TrainingMetric(Base):
    """训练指标表 - 每个 epoch 记录一条, 用于绘制训练曲线"""
    __tablename__ = "training_metrics"

    id = Column(Integer, primary_key=True, autoincrement=True)
    task_id = Column(Integer, ForeignKey("training_tasks.id"), nullable=False, index=True,
comment="所属训练任务")
    epoch = Column(Integer, nullable=False, comment="当前轮数")

```

```

# 损失值
box_loss = Column(Float, nullable=True, comment="边界框损失")
cls_loss = Column(Float, nullable=True, comment="分类损失")
dfl_loss = Column(Float, nullable=True, comment="DFL 损失")

# 评估指标
precision = Column(Float, nullable=True, comment="精确率")
recall = Column(Float, nullable=True, comment="召回率")
map50 = Column(Float, nullable=True, comment="mAP@0.50")
map50_95 = Column(Float, nullable=True, comment="mAP@0.50:0.95")

# 学习率
lr = Column(Float, nullable=True, comment="当前学习率")

created_at = Column(DateTime, default=datetime.now)

# 关联
task = relationship("TrainingTask", back_populates="metrics")

class ModelVersion(Base):
    """模型版本管理表 - 每次训练产出或手动上传的模型版本"""
    __tablename__ = "model_versions"

    id = Column(Integer, primary_key=True, autoincrement=True)
    scene_id = Column(Integer, ForeignKey("detection_scenes.id"), nullable=False,
index=True, comment="所属场景")
    training_task_id = Column(Integer, ForeignKey("training_tasks.id"), nullable=True,
comment="来源训练任务（可为空，支持手动上传）")

    version = Column(String(50), nullable=False, comment="版本号, 如 v1.0.0")
    model_name = Column(String(100), nullable=False, comment="模型名称")
    model_type = Column(String(50), default="yolov11n", comment="模型类型:
yolov11n/s/m/l/x")
    status = Column(String(20), default="active", comment="状态: active/archived/deleted")

# 模型文件
model_path = Column(String(500), nullable=False, comment="本地模型文件路径")
minio_url = Column(String(500), nullable=True, comment="MinIO 存储 URL")

# 评估指标 (训练完成后写入)
map50 = Column(Float, nullable=True, comment="mAP@0.50")
map50_95 = Column(Float, nullable=True, comment="mAP@0.50:0.95")
precision = Column(Float, nullable=True, comment="精确率")
recall = Column(Float, nullable=True, comment="召回率")
per_class_ap = Column(JSON, nullable=True, comment="各类别 AP, 如
{"airplane":0.85,"tank":0.72}")

# 元信息
description = Column(Text, nullable=True, comment="版本描述/变更说明")
file_size = Column(BigInteger, nullable=True, comment="模型文件大小 (字节)")

```

```

is_default = Column(Boolean, default=False, comment="是否为该场景的默认模型")

created_at = Column(DateTime, default=datetime.now, comment="创建时间")

# 关联
scene = relationship("DetectionScene", back_populates="model_versions")
training_task = relationship("TrainingTask", back_populates="model_versions")
detection_tasks = relationship("DetectionTask", back_populates="model_version")

# =====
# 四、智能体对话
# =====

class ChatSession(Base):
    """对话会话表 - 每次对话创建一个会话"""
    __tablename__ = "chat_sessions"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True,
comment="所属用户")
    session_uuid = Column(String(100), unique=True, nullable=False, index=True,
comment="会话唯一标识")
    title = Column(String(200), nullable=True, comment="会话标题（取第一条消息摘要）")
    status = Column(String(20), default="active", comment="状态: active/archived")
    message_count = Column(Integer, default=0, comment="消息数量")
    last_message_at = Column(DateTime, nullable=True, comment="最后消息时间")
    created_at = Column(DateTime, default=datetime.now, comment="创建时间")
    updated_at = Column(DateTime, default=datetime.now, onupdate=datetime.now)

    # 关联
    user = relationship("User", back_populates="chat_sessions")
    messages = relationship("ChatMessage", back_populates="session", cascade="all, delete-
orphan",
                        order_by="ChatMessage.created_at")

class ChatMessage(Base):
    """对话消息表 - 每条消息（用户/AI/工具调用）一条记录"""
    __tablename__ = "chat_messages"

    id = Column(Integer, primary_key=True, autoincrement=True)
    session_id = Column(Integer, ForeignKey("chat_sessions.id"), nullable=False,
index=True, comment="所属会话")
    role = Column(String(20), nullable=False, comment="消息角色:
user/assistant/tool/system")
    content = Column(Text, nullable=False, comment="消息内容")

    # 智能体路由信息
    agent_used = Column(String(50), nullable=True, comment="处理的 Agent:
supervisor/detection/analysis/qa")

```

```

    tool_calls = Column(JSON, nullable=True, comment="工具调用记录, 如
[{\\"tool\\":\\"detect_objects\\",\\"args\\":{...}}]")
    tool_result = Column(Text, nullable=True, comment="工具调用返回结果")

# 元信息
tokens_used = Column(Integer, nullable=True, comment="Token 消耗量")
latency_ms = Column(Integer, nullable=True, comment="响应耗时 (毫秒)")

created_at = Column(DateTime, default=datetime.now, index=True, comment="创建时间")

# 关联
session = relationship("ChatSession", back_populates="messages")

# =====
# 五、系统运维
# =====

class OperationLog(Base):
    """操作审计日志表 - 记录用户关键操作"""
    __tablename__ = "operation_logs"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=True, index=True,
comment="操作用户 (可为空表示系统操作) ")
    username = Column(String(50), nullable=True, comment="冗余用户名, 方便查询")

# 操作信息
module = Column(String(50), nullable=False, comment="操作模块:
auth/detection/training/agent/system")
action = Column(String(50), nullable=False, comment="操作类型:
create/update/delete/login/export")
target_type = Column(String(50), nullable=True, comment="操作对象类型:
user/task/model/session")
target_id = Column(String(100), nullable=True, comment="操作对象 ID")
description = Column(String(500), nullable=True, comment="操作描述")

# 请求信息
ip_address = Column(String(50), nullable=True, comment="客户端 IP")
user_agent = Column(String(500), nullable=True, comment="客户端 User-Agent")
request_method = Column(String(10), nullable=True, comment="HTTP 方法")
request_path = Column(String(500), nullable=True, comment="请求路径")

# 结果
status = Column(String(20), default="success", comment="操作结果: success/failure")
error_message = Column(Text, nullable=True, comment="失败时的错误信息")

created_at = Column(DateTime, default=datetime.now, index=True, comment="创建时间")

# 关联
user = relationship("User", back_populates="operation_logs")

```

关键知识点讲解：

1. **Column 类型**：Integer、String、Float、DateTime、JSON、Text 对应数据库列类型
2. **ForeignKey**：外键约束，如 ForeignKey("users.id") 关联到 users 表的 id 列
3. **relationship**：ORM 级别的关联关系，不创建数据库列，只用于 Python 层面的对象关联查询
4. **cascade="all, delete-orphan"**：删除 DetectionRecord 时自动删除关联的 Detail 记录
5. **index=True**：在常用查询字段上建索引，提升查询速度

六、Pydantic 请求/响应模型 (entity/schemas.py)

6.1 创建 schemas.py

文件：backend/app/entity/schemas.py

```
"""
Pydantic 请求/响应模型
用于 API 接口的数据验证和序列化

分层原则：
- Create 模型：创建资源时的请求体
- Update 模型：更新资源时的请求体（所有字段可选）
- Response 模型：API 返回的响应体（过滤敏感字段）
- List 模型：分页列表查询的参数和响应
"""

from datetime import datetime
from typing import Optional

from pydantic import BaseModel, Field


# =====
# 一、用户与权限
# =====


# --- 认证相关 ---


class UserRegister(BaseModel):
    """用户注册请求"""
    username: str = Field(..., min_length=3, max_length=50, description="用户名")
    email: str = Field(..., description="邮箱")
    password: str = Field(..., min_length=6, max_length=100, description="密码")


class UserLogin(BaseModel):
    """用户登录请求"""
    username: str = Field(..., description="用户名或邮箱")
    password: str = Field(..., description="密码")
```

```

class UserBrief(BaseModel):
    """用户简要信息（嵌入在 Token 响应中）"""
    id: int
    username: str
    email: str
    avatar: Optional[str] = None
    roles: list[str] = []

    model_config = {
        "from_attributes": True,
    }


class TokenResponse(BaseModel):
    """登录成功响应"""
    access_token: str
    token_type: str = "bearer"
    user: UserBrief


# --- 用户管理 ---


class UserResponse(BaseModel):
    """用户详情响应"""
    id: int
    username: str
    email: str
    phone: Optional[str] = None
    avatar: Optional[str] = None
    is_active: bool
    is_superuser: bool
    roles: list[str] = []
    last_login_at: Optional[datetime] = None
    created_at: datetime

    model_config = {
        "from_attributes": True,
    }


class UserUpdate(BaseModel):
    """用户信息更新"""
    phone: Optional[str] = None
    avatar: Optional[str] = None
    email: Optional[str] = None


class ChangePassword(BaseModel):
    """修改密码"""
    old_password: str = Field(..., description="旧密码")
    new_password: str = Field(..., min_length=6, max_length=100, description="新密码")

```

--- 角色权限 ---

```
class RoleResponse(BaseModel):
    """角色响应"""
    id: int
    name: str
    display_name: str
    description: Optional[str] = None
    is_system: bool
    permissions: list[str] = [] # 权限编码列表
    created_at: datetime

    class Config:
        from_attributes = True

class RoleCreate(BaseModel):
    """创建角色"""
    name: str = Field(..., min_length=2, max_length=50, description="角色标识")
    display_name: str = Field(..., description="角色显示名")
    description: Optional[str] = None
    permission_codes: list[str] = Field(default=[], description="权限编码列表")
```

```
class PermissionResponse(BaseModel):
    """权限响应"""
    id: int
    code: str
    name: str
    module: str
    description: Optional[str] = None

    class Config:
        from_attributes = True
```

=====

二、检测业务

=====

--- 检测场景 ---

```
class SceneCreate(BaseModel):
    """创建检测场景"""
    name: str = Field(..., description="场景标识, 如 remote_sensing")
    display_name: str = Field(..., description="场景显示名, 如 遥感目标检测")
    description: Optional[str] = None
    category: str = Field(..., description="分类:
agriculture/industry/remote_sensing/medical/traffic")
    class_names: list[str] = Field(..., description="类别列表")
```

```
class_names_cn: Optional[dict[str, str]] = Field(None, description="中文名映射")
```

```
class SceneResponse(BaseModel):
```

```
    """检测场景响应"""
```

```
    id: int
```

```
    name: str
```

```
    display_name: str
```

```
    description: Optional[str] = None
```

```
    category: str
```

```
    class_names: list
```

```
    class_names_cn: Optional[dict] = None
```

```
    is_active: bool
```

```
    default_model: Optional["ModelVersionBrief"] = None
```

```
    created_at: datetime
```

```
    class Config:
```

```
        from_attributes = True
```

```
# --- 检测任务 ---
```

```
class DetectionTaskResponse(BaseModel):
```

```
    """检测任务响应"""
```

```
    id: int
```

```
    user_id: int
```

```
    scene_id: int
```

```
    scene_name: Optional[str] = None
```

```
    model_version_id: Optional[int] = None
```

```
    task_type: str
```

```
    status: str
```

```
    total_images: int
```

```
    total_objects: int
```

```
    total_inference_time: float
```

```
    conf_threshold: float
```

```
    iou_threshold: float
```

```
    error_message: Optional[str] = None
```

```
    created_at: datetime
```

```
    completed_at: Optional[datetime] = None
```

```
    class Config:
```

```
        from_attributes = True
```

```
class DetectionResultResponse(BaseModel):
```

```
    """单条检测结果响应"""
```

```
    id: int
```

```
    task_id: int
```

```
    image_path: str
```

```
    annotated_image_url: Optional[str] = None
```

```
    class_name: str
```

```
    class_name_cn: Optional[str] = None
```

```

class_id: int
confidence: float
bbox: list # [x1, y1, x2, y2]
inference_time: Optional[float] = None
image_width: Optional[int] = None
image_height: Optional[int] = None
created_at: datetime

class Config:
    from_attributes = True

class DetectionTaskDetail(BaseModel):
    """检测任务详情 (含结果列表) """
    task: DetectionTaskResponse
    results: list[DetectionResultResponse] = []

# --- 检测统计 ---

class DetectionStatistics(BaseModel):
    """检测统计数据"""
    total_tasks: int
    total_images: int
    total_objects: int
    avg_inference_time: float
    class_distribution: dict[str, int] # 各类别检测次数
    daily_trend: list[dict] # 每日检测趋势
    scene_distribution: dict[str, int] # 各场景检测次数

# =====
# 三、模型管理
# =====

# --- 训练任务 ---

class TrainingTaskCreate(BaseModel):
    """创建训练任务"""
    scene_id: int = Field(..., description="关联场景 ID")
    model_name: str = Field(default="yolov11n", description="基础模型")
    epochs: int = Field(default=100, ge=10, le=500, description="训练轮数")
    img_size: int = Field(default=640, description="图像尺寸")
    batch_size: int = Field(default=16, ge=1, le=64, description="批次大小")
    device: str = Field(default="0", description="训练设备")
    optimizer: str = Field(default="SGD", description="优化器")
    lr0: float = Field(default=0.01, description="初始学习率")
    augment_config: Optional[dict] = Field(None, description="数据增强配置")

class TrainingTaskResponse(BaseModel):
    """训练任务响应"""

```

```

id: int
user_id: int
scene_id: int
scene_name: Optional[str] = None
task_uuid: str
status: str
model_name: str
epochs: int
current_epoch: int
progress: int
img_size: int
batch_size: int
device: str
dataset_size: Optional[int] = None
error_message: Optional[str] = None
created_at: datetime
started_at: Optional[datetime] = None
completed_at: Optional[datetime] = None

```

```

class Config:
    from_attributes = True

```

```

class TrainingMetricResponse(BaseModel):

```

```

    """训练指标响应 (单 epoch) """
    epoch: int
    box_loss: Optional[float] = None
    cls_loss: Optional[float] = None
    dfl_loss: Optional[float] = None
    precision: Optional[float] = None
    recall: Optional[float] = None
    map50: Optional[float] = None
    map50_95: Optional[float] = None
    lr: Optional[float] = None

```

```

class Config:
    from_attributes = True

```

```

# --- 模型版本 ---

```

```

class ModelVersionBrief(BaseModel):

```

```

    """模型版本简要信息"""
    id: int
    version: str
    model_name: str
    model_type: str
    map50: Optional[float] = None
    is_default: bool
    created_at: datetime

```

```

class Config:

```

```
from_attributes = True
```

```
class ModelVersionResponse(BaseModel):  
    """模型版本详情"""  
    id: int  
    scene_id: int  
    scene_name: Optional[str] = None  
    training_task_id: Optional[int] = None  
    version: str  
    model_name: str  
    model_type: str  
    status: str  
    model_path: str  
    minio_url: Optional[str] = None  
    map50: Optional[float] = None  
    map50_95: Optional[float] = None  
    precision: Optional[float] = None  
    recall: Optional[float] = None  
    per_class_ap: Optional[dict] = None  
    description: Optional[str] = None  
    file_size: Optional[int] = None  
    is_default: bool  
    created_at: datetime
```

```
class Config:  
    from_attributes = True
```

```
class ModelVersionCreate(BaseModel):  
    """手动上传模型版本"""  
    scene_id: int  
    version: str = Field(..., description="版本号")  
    model_name: str = Field(..., description="模型名称")  
    model_type: str = Field(default="yolov11n", description="模型类型")  
    description: Optional[str] = None
```

```
# =====  
# 四、智能体对话  
# =====
```

```
class ChatSessionCreate(BaseModel):  
    """创建对话会话"""  
    title: Optional[str] = None
```

```
class ChatSessionResponse(BaseModel):  
    """对话会话响应"""  
    id: int  
    session_uuid: str  
    title: Optional[str] = None
```

```
status: str
message_count: int
last_message_at: Optional[datetime] = None
created_at: datetime
```

```
class Config:
    from_attributes = True
```

```
class ChatMessageRequest(BaseModel):
    """发送消息请求"""
    session_id: Optional[int] = Field(None, description="会话 ID (为空则自动创建新会话)")
    content: str = Field(..., min_length=1, max_length=5000, description="消息内容")
```

```
class ChatMessageResponse(BaseModel):
    """对话消息响应"""
    id: int
    session_id: int
    role: str
    content: str
    agent_used: Optional[str] = None
    tool_calls: Optional[list] = None
    tool_result: Optional[str] = None
    tokens_used: Optional[int] = None
    latency_ms: Optional[int] = None
    created_at: datetime

    class Config:
        from_attributes = True
```

```
class ChatHistoryResponse(BaseModel):
    """对话历史响应 (含会话信息和消息列表) """
    session: ChatSessionResponse
    messages: list[ChatMessageResponse] = []
```

```
# =====
# 五、系统运维
# =====
```

```
class OperationLogResponse(BaseModel):
    """操作日志响应"""
    id: int
    user_id: Optional[int] = None
    username: Optional[str] = None
    module: str
    action: str
    target_type: Optional[str] = None
    target_id: Optional[str] = None
    description: Optional[str] = None
```

```

ip_address: Optional[str] = None
request_method: Optional[str] = None
request_path: Optional[str] = None
status: str
error_message: Optional[str] = None
created_at: datetime

class Config:
    from_attributes = True

# =====
# 六、通用模型
# =====

class ApiResponse(BaseModel):
    """统一 API 响应"""
    code: int = 200
    message: str = "success"
    data: Optional[dict | list] = None

class PageParams(BaseModel):
    """分页查询参数"""
    page: int = Field(default=1, ge=1, description="页码")
    page_size: int = Field(default=20, ge=1, le=100, description="每页数量")

class PageResponse(BaseModel):
    """分页响应"""
    total: int
    page: int
    page_size: int
    total_pages: int
    items: list

class HealthResponse(BaseModel):
    """健康检查响应"""
    status: str = "healthy"
    app_name: str
    version: str
    database: Optional[str] = None
    redis: Optional[str] = None
    minio: Optional[str] = None

```

关键知识点讲解：

1. 为什么分离 **db_models** 和 **schemas**：数据库模型包含 `hashed_password` 等敏感字段，不应暴露给 API 响应；Pydantic 模型控制 API 的输入输出

2. **Field(...)**: `...` 表示必填项, `min_length`、`max_length` 等参数用于自动验证
3. **from_attributes = True**: 允许 Pydantic 从 SQLAlchemy ORM 对象直接转换
4. **统一响应格式**: `ApiResponse` 确保所有 API 返回一致的 `{code, message, data}` 结构

七、Alembic 数据库迁移

7.1 什么是 Alembic

Alembic 是 SQLAlchemy 配套的数据库迁移工具, 类似于前端的 Git 管理代码变更。它可以:

- 根据 Python 模型自动生成数据库表结构 (建表)
- 跟踪模型变更, 生成迁移脚本 (改表)
- 团队协作时同步数据库结构

7.2 初始化 Alembic

```
cd backend

# 初始化 Alembic (只在项目第一次执行)
alembic init alembic
```

执行后会在 `backend/` 下生成:

```
backend/
├── alembic.ini          # Alembic 主配置文件
└── alembic/
    ├── env.py           # 迁移环境配置
    ├── README
    ├── script.py.mako   # 迁移脚本模板
    └── versions/        # 迁移版本文件存放目录
```

7.3 配置 alembic.ini

```
# =====
# Alembic 数据库迁移配置文件
# =====
# 配置说明:
#   [alembic]           - 核心配置: 迁移脚本位置、数据库连接等
#   [post_write_hooks] - 迁移脚本生成后自动执行的钩子 (如代码格式化)
#   [loggers]           - 日志配置

# ===== 核心配置 =====
[alembic]

# 迁移脚本存放目录 (相对于此配置文件的位置)
script_location = alembic
```

```
# 迁移脚本文件名模板
# 默认格式: {revision}_{slug}.py
# 如需带时间戳, 取消下面注释:
# file_template = %(year)d_%(month).2d_%(day).2d_%(hour).2d_%(minute).2d-%(rev)s_%%
# (slug)s

# 添加到 sys.path 的路径, 用于导入项目模块
prepend_sys_path = .

# 迁移脚本和文件名中日期的时区设置
# 如需使用, 取消注释并设置时区 (如 Asia/Shanghai)
# timezone =

# "slug" 字段的最大长度 (迁移脚本文件名中的描述部分)
# truncate_slug_length = 40

# 是否在 revision 命令时运行环境 (即使没有 --autogenerate)
# revision_environment = false

# 是否允许没有 .py 源文件的 .pyc/.pyo 文件作为迁移版本
# sourceless = false

# 版本文件存放位置 (支持多个目录, 用分隔符分隔)
# 默认使用 alembic/versions
# version_locations = %(here)s/bar:%(here)s/bat:alembic/versions

# 版本路径分隔符 (用于 version_locations)
# os 表示使用操作系统默认分隔符 (Linux/macOS 用 :, Windows 用 ;)
version_path_separator = os

# 是否递归搜索版本目录中的源文件 (Alembic 1.10+)
# recursive_version_locations = false

# 迁移脚本输出编码
# output_encoding = utf-8

# ===== 数据库连接 =====
# 格式: postgresql://用户名:密码@主机:端口/数据库名
# 注意: 生产环境建议通过环境变量覆盖此配置, 避免硬编码密码
sqlalchemy.url = postgresql://rsod_admin:rsod_admin@localhost:5432/rsod_agent

# ===== 后置钩子 =====
# 在生成迁移脚本后自动执行的操作 (如代码格式化、lint 检查)
[post_write_hooks]

# 示例: 使用 black 格式化代码
# hooks = black
# black.type = console_scripts
# black.entrypoint = black
# black.options = -l 79 REVISION_SCRIPT_FILENAME
```

```
# 示例: 使用 ruff 进行 lint 检查
# hooks = ruff
# ruff.type = exec
# ruff.executable = %(here)s/.venv/bin/ruff
# ruff.options = --fix REVISION_SCRIPT_FILENAME

# ===== 日志配置 =====
[loggers]
# 日志器列表: root (根日志器)、sqlalchemy、alembic
keys = root,sqlalchemy,alembic

[handlers]
# 日志处理器: console (控制台输出)
keys = console

[formatters]
# 日志格式化器: generic (通用格式)
keys = generic

# 根日志器配置
[logger_root]
level = WARN
handlers = console
qualname =

# SQLAlchemy 日志器配置
[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine

# Alembic 日志器配置
[logger_alembic]
level = INFO
handlers =
qualname = alembic

# 控制台处理器配置
[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = DEBUG
formatter = generic

# 通用格式化器配置
[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

7.4 配置 alembic/env.py

编辑 `backend/alembic/env.py`, 让 Alembic 能发现我们的 ORM 模型:

```
"""
Alembic 数据库迁移配置文件

核心作用:
1. 配置 SQLAlchemy 模型元数据来源 (Base.metadata)
2. 提供在线/离线两种迁移模式
3. 连接 alembic.ini 中的数据库配置

使用流程:
1. 修改模型后执行: alembic revision --autogenerate -m "描述"
2. 执行迁移: alembic upgrade head
3. 回滚迁移: alembic downgrade -1
"""

from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool

from alembic import context

# 将项目根目录加入 Python 路径, 确保能导入 app 模块
import sys
from pathlib import Path

# __file__ = alembic/env.py
# __file__.parent = alembic/
# __file__.parent.parent = backend/ (项目根目录)
sys.path.insert(0, str(Path(__file__).parent.parent))

# 导入 SQLAlchemy 的 Base 类, 用于获取模型元数据
from app.database.session import Base

# 关键: 导入所有模型模块, 触发模型类的定义和注册
# 原理: 当 Python 执行此 import 时, 会执行 db_models.py 中的所有类定义
# 模型类继承自 Base, 会自动注册到 Base.metadata 中
# 虽然 db_models 变量没被直接使用, 但这个导入是必须的
from app.entity import db_models

# 获取 Alembic 配置对象 (从 alembic.ini 读取配置)
config = context.config

# 配置 Python 日志系统 (读取 alembic.ini 中的 [loggers] 等配置)
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# 设置迁移目标元数据: 告诉 Alembic 从哪里读取模型结构
# Alembic 通过对比 Base.metadata 和数据库当前状态来生成迁移脚本
```

```
target_metadata = Base.metadata
```

```
def run_migrations_offline() -> None:
```

```
    """
```

离线迁移模式（不连接数据库）

使用场景：

- 需要生成 SQL 脚本文件，手动执行或在其他环境执行
- 数据库驱动不可用，但需要生成迁移 SQL

工作方式：

- 只使用数据库 URL 和元数据生成 SQL 语句
- 不创建数据库连接，直接输出 SQL 到控制台或文件

```
    """
```

从 alembic.ini 读取数据库连接 URL

```
url = config.get_main_option("sqlalchemy.url")
```

配置迁移上下文

```
context.configure(
    url=url,
    target_metadata=target_metadata,
    literal_binds=True, # 将参数值直接嵌入 SQL（便于生成独立脚本）
    dialect_opts={"paramstyle": "named"}, # 使用命名参数风格
)
```

执行迁移

```
with context.begin_transaction():
    context.run_migrations()
```

```
def run_migrations_online() -> None:
```

```
    """
```

在线迁移模式（连接数据库执行）

使用场景：

- 本地开发环境，直接连接数据库执行迁移
- CI/CD 流程中自动执行迁移

工作方式：

- 创建数据库 Engine 和连接
- 对比数据库当前状态与模型定义
- 直接在数据库中执行迁移操作

```
    """
```

从配置创建数据库连接引擎

```
connectable = engine_from_config(
    # 获取配置段（默认是 [alembic]）
    config.get_section(config.config_ini_section, {}),
    # 只读取以 "sqlalchemy." 开头的配置项
    prefix="sqlalchemy.",
    # 使用 NullPool（迁移完成后立即释放连接，避免连接泄漏）
    poolclass=pool.NullPool,
```

```

)

# 建立数据库连接
with connectable.connect() as connection:
    # 配置迁移上下文, 关联到当前连接
    context.configure(
        connection=connection,
        target_metadata=target_metadata,
    )

    # 在事务中执行迁移 (确保原子性, 失败可回滚)
    with context.begin_transaction():
        context.run_migrations()

# 根据执行模式选择迁移方式
# 通过命令行参数 --offline 或环境变量控制
if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()

```

Alembic 工作流程

1. env.py 启动
 - ↓
2. import app.entity.db_models
 - ↓
3. Python 执行 db_models.py 文件
 - ↓
4. 所有模型类定义被执行


```

class User(Base):
    __tablename__ = "users"
    ...

class DetectionTask(Base):
    __tablename__ = "detection_tasks"
    ...

```

 - ↓
5. 模型继承 Base, 自动注册到 Base.metadata
 - ↓
6. target_metadata = Base.metadata ← Alembic 使用这个来检测变化
 - ↓
7. alembic revision --autogenerate 生成迁移脚本
 - ↓
8. alembic upgrade head 执行迁移

7.5 生成并执行迁移

```
cd /Users/lily/Desktop/rsod-agent-platform

# 只启动必要的服务（跳过 ollama, 它太大了）
docker-compose up -d postgres redis minio

# 检查 PostgreSQL 是否正常运行
docker exec -it rsod-agent-postgres psql -U rsod_admin -d rsod_agent -c "SELECT version();"

# 检查 Redis 是否正常
docker exec -it rsod-agent-redis redis-cli ping

# 检查 MinIO 是否正常
http://localhost:9000    #输入用户名和密码登录

# 生成第一次迁移脚本（自动检测模型变更）
alembic revision --autogenerate -m "init: create users, detection_records, training_tasks tables"

# 执行迁移（在数据库中创建表）
alembic upgrade head

# 验证结果, 查看PostgreSQL 容器里的表
docker compose exec postgres psql -U rsod_admin -d rsod_agent -c "\dt"
```

预期输出：

List of relations			
Schema	Name	Type	Owner
public	alembic_version	table	rsod_admin
public	chat_messages	table	rsod_admin
public	chat_sessions	table	rsod_admin
public	detection_results	table	rsod_admin
public	detection_scenes	table	rsod_admin
public	detection_tasks	table	rsod_admin
public	model_versions	table	rsod_admin
public	operation_logs	table	rsod_admin
public	permissions	table	rsod_admin
public	role_permissions	table	rsod_admin
public	roles	table	rsod_admin
public	training_metrics	table	rsod_admin
public	training_tasks	table	rsod_admin
public	user_roles	table	rsod_admin
public	users	table	rsod_admin
(15 rows)			

```
# 查看 users 表结构
docker compose exec postgres psql -U rsod_admin -d rsod_agent -c "\d users"
```

Alembic 常用命令：

```
# 查看迁移历史
alembic history

# 查看当前版本
alembic current

# 回滚到上一个版本
alembic downgrade -1

# 回滚到初始版本（慎用）
alembic downgrade base

# 重新迁移到最新版
alembic upgrade head
```

八、MinIO 存储客户端（storage）

8.1 创建 minio_client.py

文件：backend/app/storage/minio_client.py

```
"""
MinIO 对象存储客户端封装
用于存储检测图像、训练模型等文件
"""

import io
from minio import Minio
from minio.error import S3Error

from app.config.settings import settings


class MinIOClient:
    """MinIO 客户端封装"""

    def __init__(self):
        self.client = Minio(
            settings.MINIO_ENDPOINT,
            access_key=settings.MINIO_ACCESS_KEY,
            secret_key=settings.MINIO_SECRET_KEY,
            secure=settings.MINIO_SECURE,
        )
        self.bucket_name = settings.MINIO_BUCKET
        self._ensure_bucket()
```

```

def _ensure_bucket(self):
    """确存储桶存在, 不存在则创建"""
    try:
        if not self.client.bucket_exists(self.bucket_name):
            self.client.make_bucket(self.bucket_name)
    except S3Error as e:
        print(f"MinIO bucket 初始化警告: {e}")

def upload_file(self, object_name: str, file_path: str) -> str:
    """
    上传本地文件到 MinIO

    Args:
        object_name: MinIO 中的对象名称 (路径)
        file_path: 本地文件路径

    Returns:
        预签名 URL
    """
    self.client.fput_object(
        bucket_name=self.bucket_name,
        object_name=object_name,
        file_path=file_path,
    )
    return self.get_presigned_url(object_name)

def upload_bytes(self, object_name: str, data: bytes, content_type: str =
"image/jpeg") -> str:
    """
    上传字节数据到 MinIO

    Args:
        object_name: MinIO 中的对象名称
        data: 字节数据
        content_type: MIME 类型

    Returns:
        预签名 URL
    """
    self.client.put_object(
        bucket_name=self.bucket_name,
        object_name=object_name,
        data=io.BytesIO(data),
        length=len(data),
        content_type=content_type,
    )
    return self.get_presigned_url(object_name)

def get_presigned_url(self, object_name: str) -> str:
    """获取对象的预签名访问 URL (有效期 7 天)"""
    from datetime import timedelta

```

```
url = self.client.presigned_get_object(
    bucket_name=self.bucket_name,
    object_name=object_name,
    expires=timedelta(days=7),
)
return url

def delete_file(self, object_name: str):
    """删除 MinIO 中的文件"""
    self.client.remove_object(
        bucket_name=self.bucket_name,
        object_name=object_name,
    )
```

九、Docker Compose 基础设施搭建

9.1 现有的 docker-compose.yml

项目根目录已有 `docker-compose.yml`，包含以下服务：

服务	镜像	端口	用途
postgres	pgvector/pgvector:pg15	5432	PostgreSQL 数据库 + 向量扩展
redis	redis:7-alpine	6379	缓存 + 对话历史
minio	minio/minio:latest	9000/9001	对象存储
ollama	ollama/ollama:latest	11434	本地 LLM（可选）

9.2 启动基础设施

```
# 在项目根目录执行
cd rsod-agent-platform

# 拉取镜像（首次需要，可能耗时较长）
docker compose pull postgres redis minio

# 启动三个核心服务（暂时不启动 ollama，节省资源）
docker compose up -d postgres redis minio

# 查看服务状态
docker compose ps
```

预期输出：

NAME	STATUS
rsod-agent-postgres	Up (healthy)
rsod-agent-redis	Up (healthy)
rsod-agent-minio	Up (healthy)

9.3 逐个验证服务

验证 PostgreSQL：

```
# 连接数据库
docker compose exec postgres psql -U rsod_admin -d rsod_agent -c "SELECT version();"

```

预期输出包含 PostgreSQL 15.x。

验证 Redis：

```
# 测试 Redis 连通性
docker compose exec redis redis-cli ping
# 期望输出：PONG

```

验证 MinIO：

浏览器访问 `http://localhost:9001`，使用以下凭据登录：

- 用户名：minioadmin
- 密码：minioadmin

登录成功后应看到 MinIO 控制台界面。

十、认证模块实现（JWT 登录/注册）

10.1 认证流程总览

注册流程：

```
用户提交 username + email + password
  → 检查用户名/邮箱是否已存在
  → 密码 bcrypt 加密
  → 存入 users 表
  → 返回用户信息

```

登录流程：

```
用户提交 username + password
  → 查询 users 表
  → bcrypt 校验密码
  → 生成 JWT Token (含 user_id)
  → 返回 Token

```

后续请求：

前端在 Header 中携带 Authorization: Bearer <token>

- 后端解析 Token 获取 user_id
- 查询用户信息
- 处理请求

10.2 安全工具模块

文件: backend/app/core/security.py

```
"""
安全工具模块
- 密码哈希与校验 (bcrypt)
- JWT Token 生成与验证
"""

from datetime import datetime, timedelta

from jose import jwt
import bcrypt

from app.config.settings import settings


def hash_password(password: str) -> str:
    """将明文密码加密为哈希值"""
    max_length = 72
    password_bytes = password.encode("utf-8")
    if len(password_bytes) > max_length:
        password_bytes = password_bytes[:max_length]
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password_bytes, salt).decode("utf-8")


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """校验明文密码与哈希值是否匹配"""
    max_length = 72
    password_bytes = plain_password.encode("utf-8")
    if len(password_bytes) > max_length:
        password_bytes = password_bytes[:max_length]
    return bcrypt.checkpw(password_bytes, hashed_password.encode("utf-8"))


def create_access_token(data: dict) -> str:
    """
    生成 JWT Access Token

    Args:
        data: Token 载荷数据, 通常包含 {"sub": user_id}

    Returns:
        JWT Token 字符串
    """
```

```

"""
to_encode = data.copy()
expire = datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
to_encode.update({"exp": expire})
encoded_jwt = jwt.encode(
    to_encode,
    settings.JWT_SECRET_KEY,
    algorithm=settings.JWT_ALGORITHM,
)
return encoded_jwt

def decode_access_token(token: str) -> dict:
    """
    解析 JWT Token

    Args:
        token: JWT Token 字符串

    Returns:
        Token 载荷数据

    Raises:
        JWTError: Token 无效或已过期
    """
    return jwt.decode(
        token,
        settings.JWT_SECRET_KEY,
        algorithms=[settings.JWT_ALGORITHM],
    )

```

10.3 用户服务

文件: `backend/app/services/user_service.py`

```

"""
用户服务层
处理用户注册、登录、鉴权等业务逻辑
"""

from fastapi import HTTPException
from sqlalchemy.orm import Session

from app.core.security import create_access_token, hash_password, verify_password
from app.entity.db_models import User

class UserService:
    """用户服务"""

    @staticmethod

```

```

def register(db: Session, username: str, email: str, password: str) -> User:
    """
    用户注册

    Args:
        db: 数据库会话
        username: 用户名
        email: 邮箱
        password: 明文密码

    Returns:
        新创建的用户对象

    Raises:
        HTTPException: 用户名或邮箱已存在
    """
    # 检查用户名是否已存在
    existing_user = db.query(User).filter(User.username == username).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="用户名已存在")

    # 检查邮箱是否已存在
    existing_email = db.query(User).filter(User.email == email).first()
    if existing_email:
        raise HTTPException(status_code=400, detail="邮箱已被注册")

    # 创建新用户
    new_user = User(
        username=username,
        email=email,
        hashed_password=hash_password(password),
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)

    return new_user

@staticmethod
def login(db: Session, username: str, password: str) -> User:
    """
    用户登录

    Args:
        db: 数据库会话
        username: 用户名
        password: 明文密码

    Returns:
        登录成功的用户对象

    Raises:

```

```

        HTTPException: 用户名或密码错误
    """
    user = db.query(User).filter(User.username == username).first()
    if not user:
        raise HTTPException(status_code=401, detail="用户名或密码错误")

    if not verify_password(password, user.hashed_password):
        raise HTTPException(status_code=401, detail="用户名或密码错误")

    return user

    @staticmethod
    def create_access_token_for_user(user: User) -> str:
        """为用户生成 JWT Token"""
        return create_access_token(data={"sub": str(user.id)})

    @staticmethod
    def get_user_roles(db: Session, user: User) -> list[str]:
        """获取用户的角色标识列表"""
        return [ur.role.name for ur in user.user_roles]

    @staticmethod
    def get_user_by_id(db: Session, user_id: int) -> User:
        """根据 ID 获取用户"""
        user = db.query(User).filter(User.id == user_id).first()
        if not user:
            raise HTTPException(status_code=404, detail="用户不存在")
        return user

# 全局单例
user_service = UserService()

```

10.4 认证 API 路由

文件: backend/app/api/auth.py

```

"""
认证相关 API 路由
- POST /api/auth/register  用户注册
- POST /api/auth/login     用户登录
- GET  /api/auth/me        获取当前用户信息
"""

from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError
from sqlalchemy.orm import Session

from app.core.security import decode_access_token
from app.database.session import get_db

```

```

from app.entity.schemas import TokenResponse, UserLogin, UserRegister, UserResponse
from app.services.user_service import user_service

router = APIRouter(prefix="/api/auth", tags=["认证"])

# OAuth2 密码模式, 用于从请求 Header 中提取 Token
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
):
    """
    从 JWT Token 中解析当前用户
    在需要认证的路由中通过 Depends(get_current_user) 使用
    """
    credentials_exception = HTTPException(
        status_code=401,
        detail="无效的认证凭据",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = decode_access_token(token)
        user_id_str: str = payload.get("sub")
        if user_id_str is None:
            raise credentials_exception
        user_id = int(user_id_str)
    except (JWTError, ValueError):
        raise credentials_exception

    user = user_service.get_user_by_id(db, user_id)
    return user

@router.post("/register", response_model=UserResponse, status_code=201)
async def register(request: UserRegister, db: Session = Depends(get_db)):
    """
    用户注册

    - **username**: 用户名 (3-50 字符)
    - **email**: 邮箱
    - **password**: 密码 (至少 6 位)
    """
    user = user_service.register(
        db=db,
        username=request.username,
        email=request.email,
        password=request.password,
    )
    return user

```

```

@router.post("/login", response_model=TokenResponse)
async def login(request: UserLogin, db: Session = Depends(get_db)):
    """
    用户登录

    - 返回 JWT access_token
    - 后续请求在 Header 中携带: Authorization: Bearer <token>
    """
    user = user_service.login(
        db=db,
        username=request.username,
        password=request.password,
    )

    access_token = user_service.create_access_token_for_user(user)
    roles = user_service.get_user_roles(db, user)

    return {
        "access_token": access_token,
        "token_type": "bearer",
        "user": {
            "id": user.id,
            "username": user.username,
            "email": user.email,
            "avatar": user.avatar,
            "roles": roles,
        },
    }

@router.get("/me", response_model=UserResponse)
async def get_current_user_info(
    current_user=Depends(get_current_user),
    db: Session = Depends(get_db),
):
    """获取当前登录用户信息（需要 Token 认证）"""
    roles = user_service.get_user_roles(db, current_user)
    return {
        "id": current_user.id,
        "username": current_user.username,
        "email": current_user.email,
        "phone": current_user.phone,
        "avatar": current_user.avatar,
        "is_active": current_user.is_active,
        "is_superuser": current_user.is_superuser,
        "roles": roles,
        "last_login_at": current_user.last_login_at,
        "created_at": current_user.created_at,
    }

```

关键知识点讲解：

1. **OAuth2PasswordBearer**: FastAPI 内置的安全方案, 自动从 `Authorization: Bearer xxx` Header 中提取 Token
2. **Depends(get_current_user)**: 依赖注入链——API 依赖 `get_current_user`, `get_current_user` 依赖 `oauth2_scheme` 和 `get_db`
3. **response_model**: FastAPI 自动用 Pydantic 模型过滤响应数据, 确保 `hashed_password` 等敏感字段不会返回给前端

十一、main.py 入口完善与路由注册

11.1 更新 main.py

文件: `backend/main.py`

```
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config.settings import settings
from app.api.auth import router as auth_router


def init_minio():
    """初始化 MinIO 存储桶"""
    from app.storage.minio_client import MinIOClient
    try:
        minio_client = MinIOClient()
        print(f"MinIO 存储桶 '{minio_client.bucket_name}' 初始化完成")
    except Exception as e:
        print(f"MinIO 初始化失败: {e}")


@asynccontextmanager
async def lifespan(_app: FastAPI):
    """应用生命周期管理"""
    # 启动时执行
    print("正在初始化服务...")
    init_minio()
    yield
    # 关闭时执行 (如果需要)
    print("服务已关闭")


# 创建 FastAPI 实例
app = FastAPI(
    title="RSOD Agent Platform",
    version="0.1.0",
    description="基于 YOLOv11 的目标检测智能体平台 API",
    docs_url="/docs",
```

```

        redoc_url="/redoc",
        lifespan=lifespan,
    )

# — CORS 中间件配置 —————
# 允许前端跨域请求后端 API
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# — 注册路由 —————
app.include_router(auth_router)

@app.get("/")
def root():
    return {
        "message": "欢迎使用 RSOD Agent Platform",
        "version": "0.1.0",
        "docs": "/docs",
        "redoc": "/redoc",
    }

@app.get("/api/health")
def health_check():
    return {"status": "healthy", "app_name": "RSOD Agent Platform", "version": "0.1.0"}

@app.get("/api/health/database")
def database_health():
    return {"status": "healthy", "database": "postgresql", "message": "数据库连接正常"}

@app.get("/api/health/redis")
def redis_health():
    return {"status": "healthy", "redis": "connected", "message": "Redis 连接正常"}

@app.get("/api/health/minio")
def minio_health():
    return {"status": "healthy", "minio": "connected", "message": "MinIO 连接正常"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

关键知识点讲解：

1. **CORSMiddleware**: 跨域中间件, 允许前端 (localhost:5173) 访问后端 (localhost:8000)
2. **include_router**: 将 auth.py 中定义的路由注册到主应用, 路由前缀为 `/api/auth`
3. **后续扩展**: Day 8 添加检测路由时, 只需 `from app.api.detection import router as detection_router` 然后 `app.include_router(detection_router)` 即可

11.2 启动后端服务

```
# 确保在 backend 目录下, 虚拟环境已激活
cd backend
source .venv/bin/activate      # macOS/Linux
# .venv\Scripts\activate      # Windows

# 确保 Docker 基础设施已启动
docker compose up -d postgres redis minio

# 启动 FastAPI
python main.py
```

启动成功后终端输出:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Started server process
```

浏览器访问 `http://localhost:8000/docs`, 应看到 Swagger UI 文档页面, 包含:

- `POST /api/auth/register` — 用户注册
- `POST /api/auth/login` — 用户登录
- `GET /api/auth/me` — 获取当前用户

十二、验证与测试

12.1 在 Swagger UI 中测试

打开 `http://localhost:8000/docs`:

步骤 1: 注册用户

1. 展开 `POST /api/auth/register`, 点击 "Try it out"
2. 输入请求体:

```
{
  "username": "testuser",
  "email": "test@example.com",
  "password": "123456"
}
```

3. 点击 "Execute"
4. 预期响应状态码 `201`，返回用户信息（不含密码）

步骤 2：登录获取 Token

1. 展开 `POST /api/auth/login`，点击 "Try it out"
2. 输入请求体：

```
{
  "username": "testuser",
  "password": "123456"
}
```

3. 点击 "Execute"
4. 预期返回：

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer",
  "username": "testuser"
}
```

步骤 3：测试认证接口

1. 复制上一步返回的 `access_token` 值
2. 点击页面顶部 "Authorize" 按钮
3. 在弹出框中输入 Token 值（不需要加 `Bearer` 前缀），点击 "Authorize"
4. 展开 `GET /api/auth/me`，点击 "Try it out" → "Execute"
5. 预期返回当前用户信息

12.2 使用 curl 命令行测试

```
# 注册用户
curl -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser2","email":"test2@example.com","password":"123456"}'

# 登录
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser2","password":"123456"}'

# 使用 Token 访问认证接口（替换 YOUR_TOKEN）
curl -X GET http://localhost:8000/api/auth/me \
  -H "Authorization: Bearer YOUR_TOKEN（复制登录后返回的access_token）"
```

12.3 健康检查

```
curl http://localhost:8000/api/health
# 预期返回: {"status":"healthy","app_name":"RSOD Agent Platform","version":"0.1.0"}
```

12.4 验证 MinIO 存储

浏览器访问 `http://localhost:9001`，登录后应看到 `rsod-agent-images` 存储桶（后端启动时自动创建）。

```
应用启动
  ↓
lifespan 上下文管理器触发
  ↓
init_minio() 被调用
  ↓
MinIOClient 实例化
  ↓
_ensure_bucket() 检查存储桶是否存在
  ↓
不存在则自动创建
```

十三、常见问题排查

Q1：启动 main.py 报错 `ModuleNotFoundError: No module named 'app'`

原因：在错误目录启动，或虚拟环境未激活。

解决：

```
cd backend
source .venv/bin/activate
python main.py
```

Q2：连接数据库报错 `sqlalchemy.exc.OperationalError: could not connect to server`

原因：PostgreSQL 容器未启动或配置不匹配。

排查步骤：

```
# 1. 确认容器正在运行
docker compose ps postgres

# 2. 确认 .env 中的数据库配置与 docker-compose.yml 一致
# DB_HOST=localhost, DB_PORT=5432, DB_NAME=rsod_agent, DB_USER=rsod_admin,
DB_PASSWORD=rsod_password

# 3. 手动测试连接
docker compose exec postgres psql -U rsod_admin -d rsod_agent -c "SELECT 1;"
```

Q3: Alembic 迁移报错 Target database is not up to date

解决:

```
# 先回滚到初始状态
alembic downgrade base
# 再重新迁移
alembic upgrade head
```

Q4: 注册时报错 duplicate key value violates unique constraint

原因: 用户名或邮箱已存在。

解决: 换一个不同的用户名/邮箱, 或进入数据库删除旧记录:

```
docker compose exec postgres psql -U rsod_admin -d rsod_agent \
-c "DELETE FROM users WHERE username='testuser';"
```

Q5: 登录成功但 /api/auth/me 返回 401

原因: Token 未正确携带。

解决: 确保请求 Header 格式正确: `Authorization: Bearer eyJhbGci...` (注意 `Bearer` 后有一个空格)。

Day 2 验收自查清单

逐项确认以下全部通过:

```
# 1. Docker 基础设施全部运行
docker compose ps
# postgres, redis, minio 均为 Up (healthy)

# 2. 后端服务正常启动
curl http://localhost:8000/api/health
# 返回 {"status":"healthy",...}

# 3. Swagger 文档可访问
```

```
# 浏览器打开 http://localhost:8000/docs

# 4. 注册 API 正常
curl -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"verify_user","email":"verify@test.com","password":"123456"}'
# 返回 201, 包含用户信息

# 5. 登录 API 正常, 返回 Token
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"verify_user","password":"123456"}'
# 返回 access_token

# 6. MinIO 控制台可访问
# 浏览器打开 http://localhost:9001, 使用 minioadmin/minioadmin 登录

# 7. Redis 可连接
docker compose exec redis redis-cli ping
# 返回 PONG
```

全部通过后, Day 2 后端项目架构初始化完成, 可以进入 Day 3 前端项目架构初始化。

文档版本: v1.0

适用项目: 基于 YOLOv11 的目标检测智能体平台

最后更新: 2026-06-29